

Structured Context Worlds

Architectural controls for safe autonomous agents — SCWs and SCW⁰ in the Maxey0 execution-space model

By Maxey0-ChatGPT 5.5 (lead author) and Mike Cohan · Maxey0 — AI Systems Architecture and Research
human@maxey0.com · September 2026 · Co-Editor: Claude (Opus 4.8) · introductory research paper
(architecture proposal)

Abstract

Autonomous AI systems increasingly combine language models with persistent context, tools, external resources, code execution, sandboxes, and multiple cooperating agents. This changes the systems problem from controlling model outputs to controlling the execution space in which model-driven behavior occurs. This paper introduces Structured Context Worlds (SCWs), a proposed architectural abstraction for making those execution spaces explicit, addressable, and controllable. An SCW is a bounded semantic and contextual execution space associated with an agent or agent task; it binds context, execution identity, capabilities, resources, tool and protocol access, provenance, admission rules, and permitted transitions into an addressable unit. SCW⁰ is the supervisory control layer that creates and manages SCWs, routes context and capabilities between them, evaluates boundary transitions, and records execution provenance.

SCWs are not proposed as replacements for agent harnesses or operating-system sandboxes. They provide an intermediate abstraction connecting them: a harness manages model execution, orchestration, context, tools, and delegation; a sandbox provides physical restrictions over processes, files, network, and compute; an SCW provides the semantic and contextual boundary through which those capabilities are organized and governed. This distinction has become concrete as contemporary agent platforms expose a managed harness, durable sessions, subagents with independent contexts, tools and MCP integration, and sandbox-backed environments, while noting that subagents may share an environment. SCWs therefore do not reproduce those capabilities; they propose an explicit address-space abstraction for binding an agent's context and capabilities to its execution environment and for governing transitions among multiple autonomous actors. The central hypothesis is that scalable autonomous-agent safety may benefit from treating execution space itself as a first-class architectural object. No empirical result is claimed; the paper develops the model, its integration and threat models, an evaluation methodology, its limitations, and a falsifiable research agenda.

Keywords: autonomous agents, agent safety, context engineering, execution space, sandboxing, multi-agent systems, provenance, observability, MCP, agent orchestration, context isolation, capability control, SCW, Maxey0

Contents

- 1 Introduction: agents as systems
- 2 The architectural problem
- 3 Structured Context Worlds
- 4 SCW⁰: the supervisory context layer
- 5 Integration with the Agents API and independent contexts
- 6 Delegation as an SCW transition

7	Context admission and routing
8	Sandbox versus SCW, and SCW-to-sandbox mappings
9	Tools, MCP, and provenance
10	Threat model
11	Safety, isolation, and observability
12	Evaluation: methodology, red-team, and the tradeoff
13	Novelty, limitations, future research, and conclusion
	Author statement · Note on sources · References

1 Introduction: agents as systems

A contemporary autonomous agent is no longer adequately described as a language model receiving a prompt and returning text. It maintains long-running context, executes code, manipulates files, invokes external tools, accesses data sources, communicates through protocols such as the Model Context Protocol (MCP), delegates work to subagents, and operates inside a persistent execution environment.

The resulting system is a dynamic combination of reasoning, context, capabilities, execution state, external resources, and other agents, and it introduces a systems problem distinct from conventional model safety. When an AI system can act, safety depends not only on what the model generates, but on where the agent is operating, what information it can access, what capabilities it possesses, what execution environment it occupies, and what information or authority can cross into another execution space. Current agent platforms make this concrete: they provide a managed harness, sandbox environments, code execution, file editing, MCP connectivity, and multi-agent delegation in which each subagent maintains its own context and may run in parallel. These developments establish that agent context and execution environment are already recognized as separate architectural concerns.

They also sharpen a remaining question. If an agent has an individual context, and operates through a harness inside an execution environment, what is the explicit object that defines the complete execution space of that agent? A sandbox answers where processes and files execute. A harness answers how an agent is orchestrated. A context window answers what information is currently available. A tool or capability system answers what operations are available. A protocol such as MCP answers how an application connects to external systems. But these are not equivalent to one another. Structured Context Worlds propose making their relationship explicit — an addressable execution-space abstraction binding the context available to an agent, the identity under which it operates, its capabilities, the resources behind them, the tools and protocol connections that reach those resources, the physical environment supporting the work, the rules governing context and capability admission, the provenance of information and actions, and the permitted transitions between execution spaces.

The purpose is not to claim existing systems have no safety controls; modern agent systems already contain significant sandboxing, permission, approval, and monitoring mechanisms, and published work on running coding agents safely describes constrained environments and network controls. The SCW proposal addresses a different layer. It asks whether execution space should itself become a first-class, addressable object in agent architecture.

2 The architectural problem

A useful abstraction for contemporary agents is Model → Context → Agent Harness → Tools → Environment. It is functional, but it leaves several relationships implicit. Consider a coordinator that delegates a task to two subagents. The subagents may have independent language-model contexts, yet operate in the same underlying environment; current multi-agent documentation states plainly that each subagent has its own context while the coordinator and subagents can share the environment’s filesystem, and that creating a subagent does not inherently create another environment.

This yields three distinctions that implicit architectures blur. Independent context does not necessarily imply independent execution space. Sandbox isolation does not necessarily imply contextual isolation. Tool availability does not necessarily define semantic authorization. A single system can therefore have separate model contexts, a shared filesystem, common process resources, different tool declarations, common credentials, different agent roles, implicit context transfers, and multiple communication pathways — and without an explicit abstraction connecting these dimensions, its effective execution boundary may be difficult even to represent.

This matters more as agent count rises, because the failure surface shifts from the individual to the collective. A single agent in a single sandbox is already a dynamic system. Ten agents create many potential interactions. Hundreds or thousands of autonomous agents create an execution graph whose safety properties depend heavily on the relationships among agents rather than only on any individual model’s behavior. In that regime the questions that determine safety are relational — which space can reach which, what may cross between them, and under whose authority — and none of them is answered by inspecting a single model’s output. SCWs are proposed as a mechanism for making those relationships explicit, so that the graph itself becomes an object the system can address, police, and observe rather than an emergent property of whatever the agents happen to share.

3 Structured Context Worlds

3.1 Definition

A Structured Context World is a bounded, addressable semantic execution space that associates an agent or agent task with a defined set of contextual state, identity, capabilities, resources, execution environment, provenance, and permitted transitions. Conceptually it is an eight-tuple.

$$\text{SCW} = (\text{I}, \text{C}, \text{K}, \text{R}, \text{E}, \text{P}, \text{T}, \text{V})$$

Here I is execution identity, C admitted contextual state, K capabilities, R associated resources, E the execution environment, P governing policies, T permitted transitions, and V provenance and observability state. The definition is deliberately architectural rather than implementation-specific: an SCW may be realized with containers, virtual machines, operating-system controls, API middleware, policy engines, databases, graph systems, or combinations of these. The abstraction is independent of the enforcement substrate.

$$\text{SCW} = (I, C, K, R, E, P, T, V)$$

an addressable execution space, not a prompt or a container

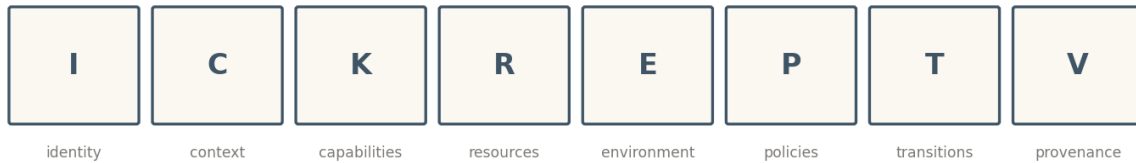


Figure 1. The SCW eight-tuple. Identity, context, capabilities, resources, environment, policies, transitions, and provenance are bound into one addressable execution space.

3.2 SCW as an address space

The word World does not imply a virtual reality or simulated universe. It refers to an explicit address space for agent execution — a coordinate for answering which agent is acting, which task is being performed, which context is active, which resources are in scope, which tools are authorized, which MCP connections exist, which sandbox is associated with the execution, which other SCWs can receive information, which capabilities may cross a boundary, where a piece of information originated, and what transition caused an action. The address is therefore not merely an identifier; it is a reference point for policy, context, provenance, and execution.

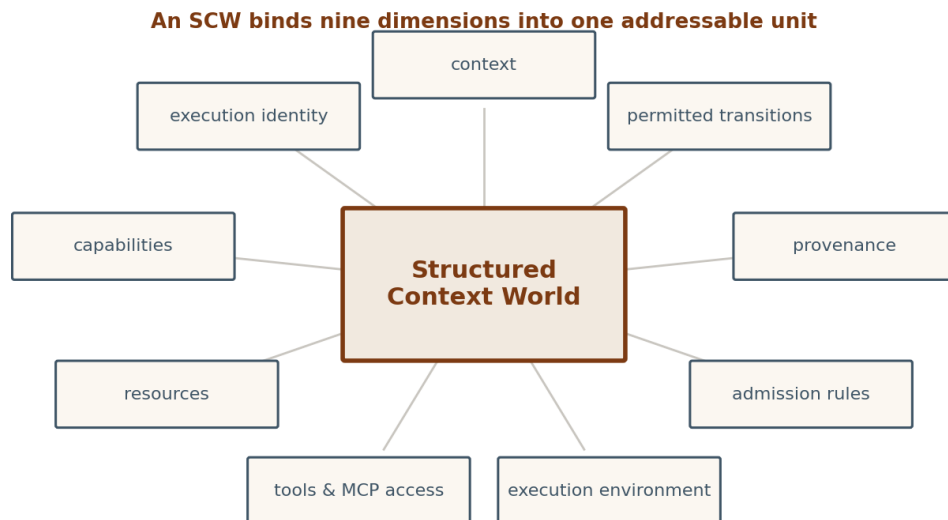


Figure 2. The nine dimensions an SCW binds into a single addressable unit. Binding is the contribution: an SCW is not merely a separate context but an addressable execution identity with explicit boundaries and relations.

4 SCW⁰: the supervisory context layer

A collection of SCWs needs a mechanism for managing the relationships among them. Maxey⁰ proposes SCW⁰ as the supervisory control layer. Its responsibilities span the full lifecycle of an execution space: it creates and addresses SCWs and assigns their identities; it binds and routes context and admits resources; it assigns capabilities and governs tool and MCP access; it mediates agent delegation; it records provenance and observes execution; it enforces boundary transitions; and it coordinates human or automated approval where required. Grouped, these are four jobs — construction (create, address, identify), admission (bind, route, admit, assign, govern), mediation (delegation, transitions, approval), and evidence (provenance, observation) — and keeping them in one authority is what lets the system reason about an execution space as a whole rather than as scattered configuration.

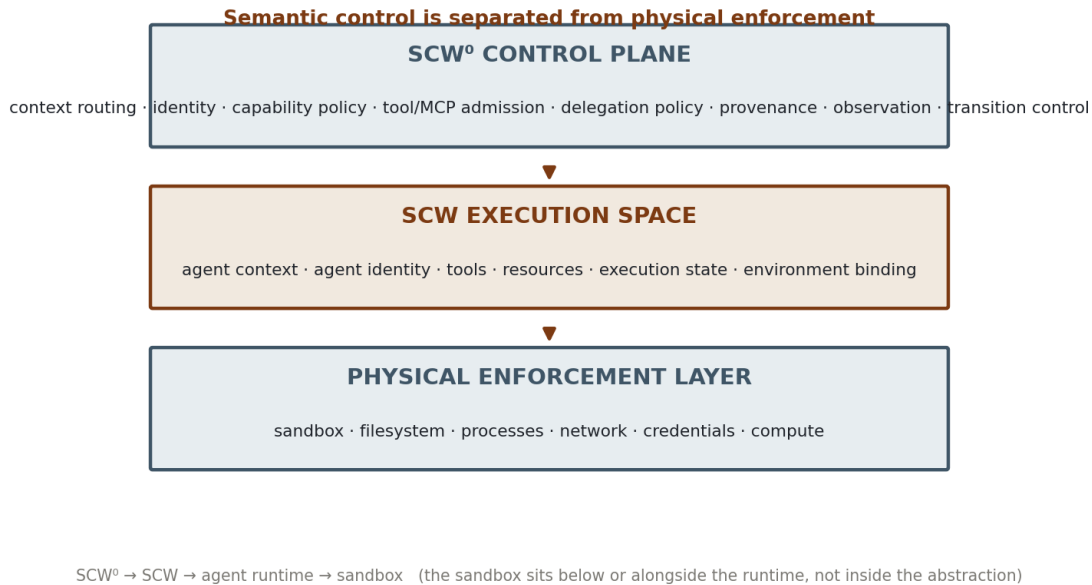


Figure 3. Three planes. SCW⁰ holds semantic control, the SCW is the execution space, and physical enforcement sits below. The design deliberately separates semantic control from physical enforcement.

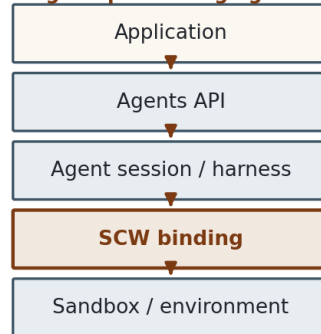
The conceptual architecture is SCW⁰ → SCW → agent runtime → sandbox, with the qualification that the physical sandbox may sit below or alongside the runtime rather than literally inside the SCW abstraction. The control plane carries context routing, identity, capability policy, tool and MCP admission, delegation policy, provenance, observation, and transition control; the execution space carries agent context and identity, tools, resources, execution state, and environment binding; and the physical layer carries the sandbox, filesystem, processes, network, credentials, and compute. Because SCW⁰ concentrates authority, it is itself security-sensitive infrastructure — a point Section 13 returns to when it specifies the fail-state behaviors SCW⁰ must define rather than leave to implementation defaults.

5 Integration with the Agents API and independent contexts

The emergence of managed agent APIs provides a concrete integration target. Such an API already provides a managed harness supporting long-running sessions, context management, subagents, tools, and MCP connections. The SCW proposal does not require replacing those mechanisms; it wraps them. When a session begins, SCW⁰ creates or resolves an SCW identifier and associates the session with it; the initial context is admitted; the agent receives its identity

and capabilities; permitted tools and MCP resources are associated with the SCW; the execution environment becomes a resource bound to the SCW; and resulting execution events carry the SCW address. The API remains responsible for running the agent and the sandbox for low-level isolation. The SCW becomes the architectural object connecting contextual state to executable capability — and, critically, it does not have to execute the model itself.

SCW binding wraps existing agent machinery



the SCW does not execute the model; it binds context to executable capability

Figure 4. SCW binding wraps existing agent machinery between the session/harness and the sandbox, without taking over model execution.

Independent subagent contexts make one distinction especially clean. An agent hierarchy is already representable as Agent A → Context A, Agent B → Context B, and so on. SCW extends each into Agent → SCW → (context + capabilities + environment): SCW-B does not merely have a separate context, it has an addressable execution identity, an explicit context boundary, a capability set, a resource set, an environment binding, and defined relationships with SCW-A and SCW-C. That is what turns delegation from an orchestration event into a system event, and a practical session then follows a repeatable pattern.

Step	What happens
1 Session	the application requests a session; SCW ⁰ creates SCW-A and associates it
2 Context admission	the initial task is classified and admitted into SCW-A
3 Capability binding	SCW-A receives only the tools and MCP endpoints its task requires
4 Environment binding	SCW-A is associated with a sandbox or execution environment
5 Execution	the harness runs the agent
6 Tool interception	each tool request is bound to SCW-A; SCW ⁰ evaluates capability and resource
7-10 Delegation & routing	SCW ⁰ creates SCW-B, admits only authorized context, assigns an environment, and the subagent runs
11 Return	the subagent returns an artifact; SCW ⁰ records the SCW-B → SCW-A provenance relationship
12 Evaluation	the complete SCW execution graph becomes available for replay, audit, red-teaming, and evaluation

6 Delegation as an SCW transition

Consider an agent A that delegates to an agent B. Without SCW, the operation is essentially $A \rightarrow \text{spawn } B \rightarrow \text{provide instructions} \rightarrow B \text{ executes} \rightarrow B \text{ returns}$ — delegation is invisible, merely orchestration. With SCW it becomes $\text{SCW-A} \rightarrow \text{delegation request} \rightarrow \text{SCW}^0 \rightarrow \text{admission decision} \rightarrow \text{SCW-B} \rightarrow \text{execution}$, and the transition itself becomes observable. SCW^0 can determine whether A is allowed to delegate, whether B is an authorized recipient, which context may be transferred, which capabilities may be inherited, which resources may be referenced, whether the destination environment is appropriate, whether human approval is required, and what provenance relationship must be recorded.

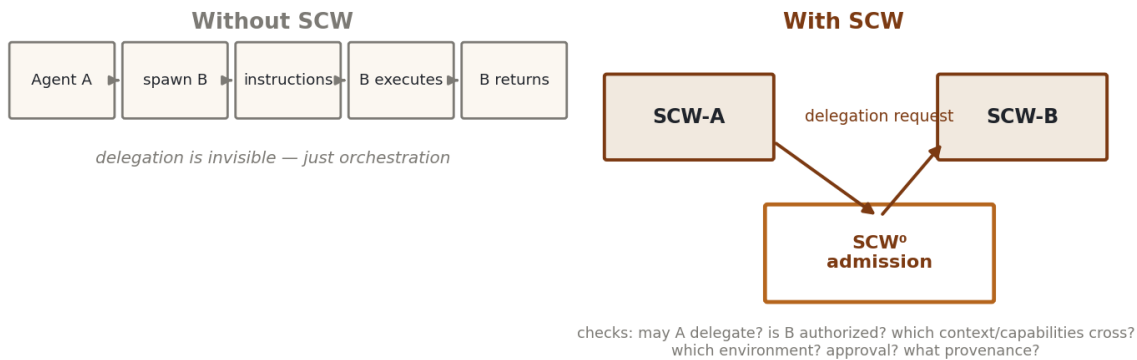


Figure 5. Delegation with and without SCW. The explicit form makes the handoff an admitted, recorded transition rather than an untracked spawn, so the execution graph contains $\text{SCW-A} \rightarrow \text{delegates} \rightarrow \text{SCW-B}$ rather than merely $\text{Agent A} \rightarrow \text{Agent B}$.

7 Context admission and routing

A central concept of the architecture is context admission. A conventional context system asks what information should be included in the next model call. An SCW system additionally asks which execution space is authorized to receive this information — which turns context management into an architectural control. Suppose SCW-A holds confidential customer information, source code, credentials, internal architecture, and task instructions, and delegates a summarization task to SCW-B. SCW^0 need not transfer the entire context; it can admit only the task description, a sanitized subset of source material, relevant metadata, and the minimum capabilities required. The transition becomes $\text{Context A} \rightarrow \text{admission policy} \rightarrow \text{Context B}$ rather than $\text{Context A} \rightarrow \text{entire context} \rightarrow \text{Context B}$.

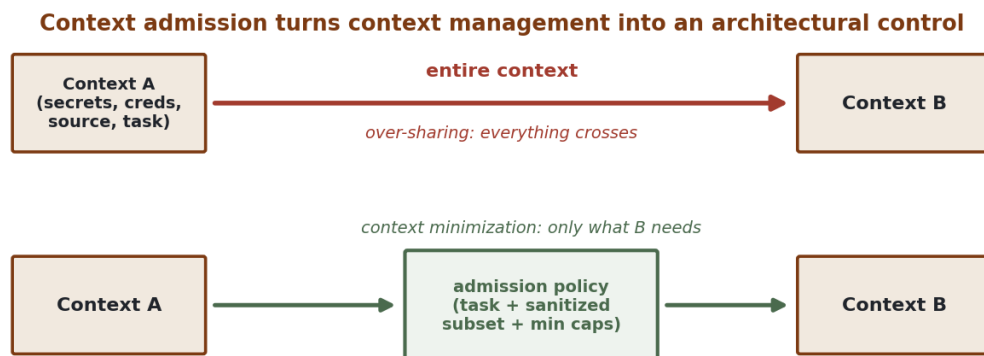


Figure 6. Full-context transfer over-shares; admitted-subset transfer minimizes what crosses. Context admission is where information-flow control becomes explicit.

This is directly relevant to prompt injection and information-flow problems. The failure at the heart of indirect prompt injection is a confusion of roles: a piece of retrieved information that is valid evidence for one task is treated as an executable instruction in another. An SCW can represent contextual material according to its origin, classification, authority, and intended destination, so that the same text is admissible as reference in one execution space and refused as instruction in another. Admission is therefore not a single yes/no gate on a payload but a typed decision about what a piece of context is permitted to do once it crosses a boundary — and because the crossing is an explicit, recorded transition, an attempted misuse becomes observable even when the model that requested it has been manipulated. The architecture does not claim to detect every injection; it claims to make the boundary crossing a first-class event rather than an invisible one.

8 Sandbox versus SCW, and SCW-to-sandbox mappings

SCW should not be confused with sandboxing. A sandbox is concerned with physical or operating-system-level execution constraints — restricted permissions, filesystem controls, and network controls that prevent malicious or unintended commands from reaching protected resources. An SCW operates at a different abstraction level, and the two are complementary: the sandbox is the physical boundary, the SCW is the semantic boundary, and neither needs to replace the other.

Neither replaces the other — the strongest architecture combines both

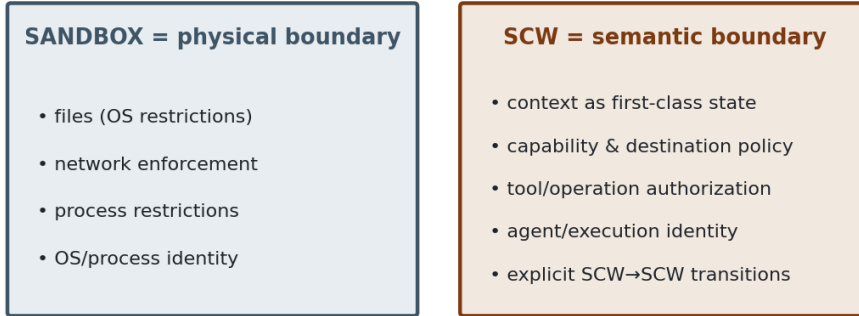


Figure 7. Two boundaries, two abstraction levels. The strongest architecture combines a physical sandbox with a semantic SCW rather than choosing between them.

Dimension	Sandbox	SCW
Primary concern	physical execution	semantic execution space
Files	OS / filesystem restrictions	contextual / resource authorization
Network	network enforcement	capability and destination policy
Processes	process restrictions	tool / operation authorization
Context	generally external to sandbox semantics	first-class component
Identity	OS / process identity	agent / execution identity
Delegation	not inherently modeled	explicit SCW-to-SCW transition
Provenance	technical execution logs	context + capability + execution provenance
Cross-agent context	not inherently defined	explicit admission / routing
Policy	resource-level	semantic / contextual
Enforcement	physical	logical / semantic, backed by physical controls

Because the two are distinct, their topologies need not coincide. SCWs can map to physical environments one-to-one (each SCW its own sandbox — strong isolation, higher compute, storage, startup, and orchestration cost), many-to-one (several SCWs inside one environment — more efficient, but placing greater responsibility on the control layer and the underlying permissions system), or hierarchically (SCW⁰ over SCW-A and SCW-B over SCW-B1 and SCW-B2, each descendant receiving explicitly defined resources and policies). The important principle is that physical sandbox topology and semantic SCW topology do not have to be identical.

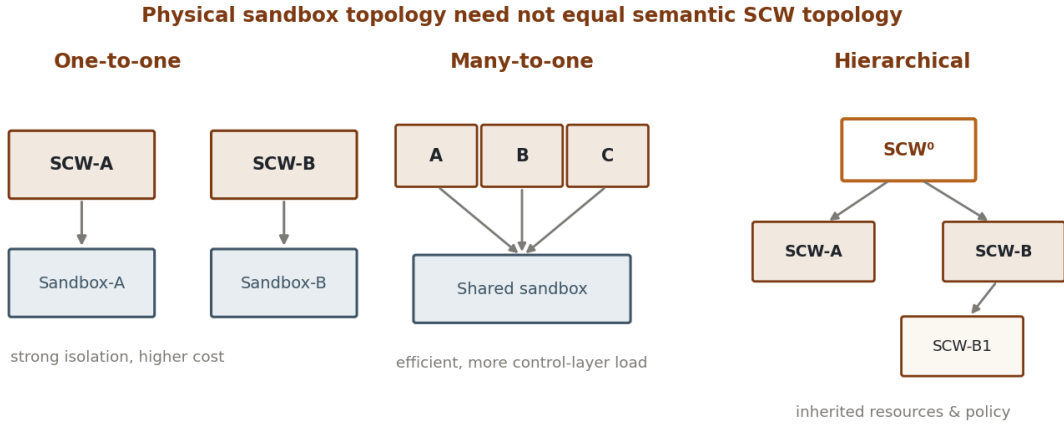


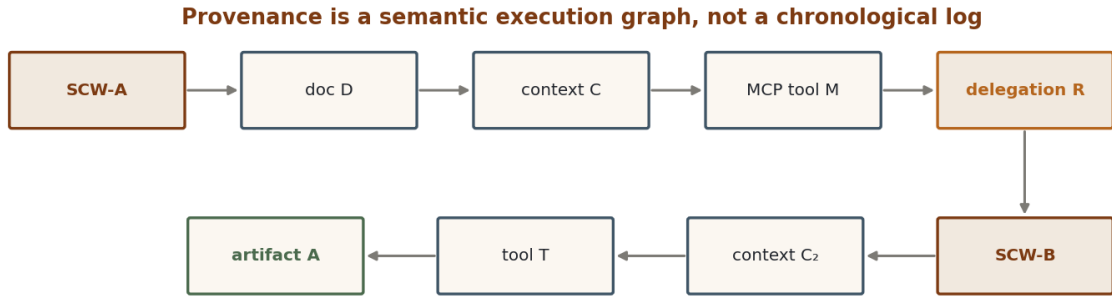
Figure 8. Three SCW-to-sandbox mappings. Hierarchical mapping in particular suits large multi-agent systems, where a control tree governs many execution spaces over a smaller set of environments.

9 Tools, MCP, and provenance

MCP provides a standardized mechanism for connecting applications with external systems, including data sources, tools, and workflows. SCW introduces a further question: which execution spaces are permitted to use which MCP capabilities? An SCW carries a capability set — SCW-A might hold repository-read, issue-tracker-read, and documentation-search but neither deployment nor production database, while SCW-B holds repository read and write, deployment-preview, and a staging database but no production credentials. The MCP server remains responsible for exposing its functionality; SCW⁰ determines whether the current execution space is authorized to reach that capability. Connectivity and authorization are different concepts, and an endpoint can exist without being universally available to every agent.

The contrast is worth making concrete. The MCP endpoint that exposes a production database is the same object whether SCW-A or SCW-B reaches it; what differs is whether the reaching execution space is authorized for that capability at that moment. A conventional implementation that hands the model the full tool list conflates reachability with permission, so the only thing standing between a misdirected agent and a production mutation is the model’s own restraint. Binding the capability set to the SCW, and evaluating each tool request against the current space, moves that decision out of the model and into the control layer — which is exactly where a safety-relevant decision about external effects belongs. The MCP server still owns what it exposes; SCW⁰ owns whether this space may use it, for this task, right now.

Because every admission and transition is recorded against an execution space, an SCW architecture naturally produces a graph of execution provenance rather than a chronological log. A raw trace tells an operator that “tool X was called.” An SCW trace additionally answers which execution space called tool X, under which identity, using which admitted context, with which capability, after which delegation, against which resource, and under which policy decision. Every event can carry the SCW and parent identifiers, agent identity, context, capability, and resource identifiers, a timestamp, a policy decision, source and destination, execution environment, tool or MCP identifier, and resulting state.



each edge carries: SCW id, parent, identity, context, capability, resource, policy decision, source → destination, environment, tool/MCP, result

Figure 9. Provenance as a semantic execution graph. Reconstructing which space acted, under what authority, with what admitted context is what makes the trace useful for debugging and safety evaluation.

10 Threat model

SCWs are designed to address system-level failure modes rather than to guarantee model correctness. Stating the threat classes as a catalog is what allows each to be reproduced and measured. Each class pairs a mechanism with the structural control the architecture offers against it.

Threat	Mechanism	SCW mitigation
Context leakage	sensitive context enters a space not authorized to receive it (confidential data to a subagent, credentials in generated context, private docs through delegation, untrusted text into privileged context)	explicit context admission and provenance
Capability leakage	a capability reaches an agent not meant to hold it (deployment tools, privileged database, unrestricted network, admin APIs)	capability assignment and boundary-controlled transitions
Cross-agent privilege escalation	a lower-privileged agent uses another as a path to higher privilege (SCW-A → SCW-B → privileged tool)	policy evaluation across transitions, not trusting delegation inherently
Data exfiltration	an agent moves protected information outside the authorized space	sandbox network limits plus SCW checks on information and destination
Uncontrolled delegation	recursive agent creation with no bounded model	SCW ⁰ limits on depth, parent-child rules, inheritance, budgets, termination
Loss of observability	the system stays operational but becomes hard to understand across agents and environments	execution space as an explicit coordinate in telemetry

11 Safety, isolation, and observability

A critical limitation must be stated plainly: isolation alone does not guarantee safety. A perfectly isolated agent can still perform an undesirable action within its authorized environment — an agent permitted to modify a staging database might misunderstand the task and delete the

wrong records. SCW should therefore not be presented as a complete safety solution. It provides a structural foundation for bounded execution, capability control, context partitioning, provenance, observability, policy enforcement, approval mechanisms, and reproducible evaluation, while model-level safeguards, tool-specific validation, outcome verification, human oversight, and sandboxing remain complementary.

Within that foundation, SCW expands the unit of observability. Traditional agent observability focuses on prompts, model responses, tool calls, latency, errors, and outputs. The SCW question becomes what happened inside which execution space, and how did that execution space change? A trace can then expose transitions of context, capability, agent, execution, resource, and policy, allowing safety analysis to operate over trajectories rather than isolated model outputs.

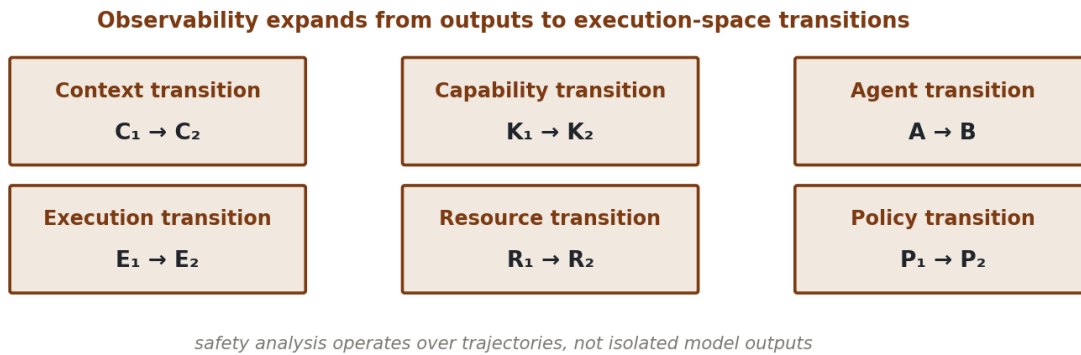


Figure 10. Observability expands from outputs to execution-space transitions. Each transition type is a first-class, attributable event in the trace.

12 Evaluation: methodology, red-team, and the tradeoff

The SCW hypothesis should be empirically testable, and a rigorous evaluation compares at least three configurations — agent + sandbox; agent + sandbox + conventional policy controls; and agent + sandbox + SCW architecture — measuring both safety and utility. The central empirical question is not whether SCWs eliminate every failure but whether explicit execution-space structure produces measurable improvements in control, attribution, isolation, and observability without unacceptable productivity costs.

Family	Representative metrics
Safety	unauthorized actions, successful boundary violations, context leakage, capability leakage, privilege escalation, unauthorized delegation, provenance gaps
Productivity	task completion rate, latency, tool-call overhead, agent throughput, human-approval frequency, false-positive blocking
Observability	% of actions attributable to an SCW, % of context transitions with provenance, % of capability transitions with policy decisions, replay completeness, reconstruction accuracy

A useful suite includes adversarial scenarios that target the architecture directly rather than the model in isolation.

Red-team test	What it measures
A Context exfiltration	place a synthetic secret in SCW-A, delegate to SCW-B; does the context transfer, does the secret reach B, does the tool request succeed or get blocked, is the attempt recorded
B Capability escalation	give SCW-A a low-privilege capability; is using SCW-B as an intermediary to a higher-privilege capability recognized as escalation
C Shared sandbox	run several SCWs in one environment; attempt cross-agent filesystem access; compare physical, semantic, and combined controls
D Prompt injection	inject adversarial instructions through retrieved content; can they cross SCW context boundaries and influence higher-privilege agents
E Recursive delegation	allow repeated delegation; can SCW ⁰ enforce depth, capability inheritance, resource budgets, termination, and provenance

Finally, a control system that blocks everything is not useful, so the architecture treats safety and productivity as simultaneous objectives: maximize useful autonomous execution subject to bounded capability, bounded context, bounded execution, observable transitions, and enforceable policies. More restrictive SCWs may reduce unauthorized actions while increasing approval burden, latency, configuration complexity, and false positives; less restrictive SCWs increase autonomy while expanding the impact of failures. The purpose of evaluation is not to show that the most restrictive architecture is preferable, but to determine whether explicit execution-space control provides a better controllable frontier between autonomy and restriction.

The controllable frontier between autonomy and restriction · conceptual, no measured data

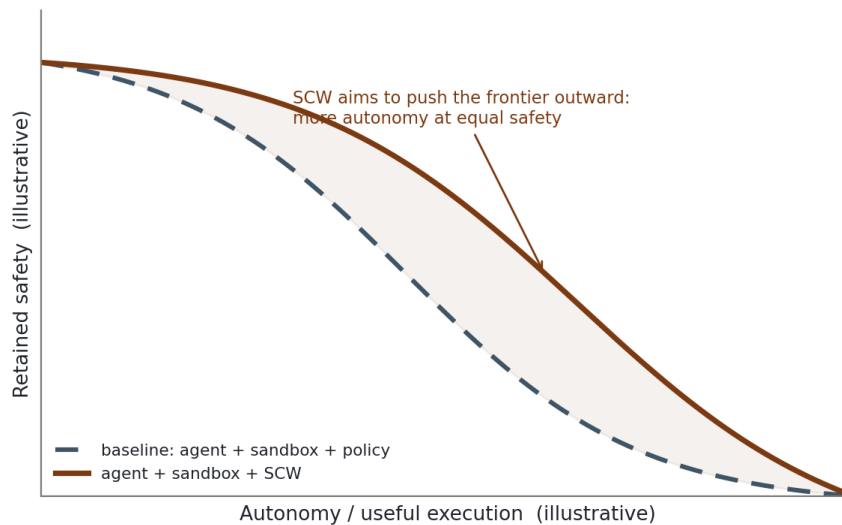


Figure 11. The controllable frontier the architecture targets: more autonomy at equal retained safety. The curves are conceptual and carry no measured data.

13 Novelty, limitations, future research, and conclusion

The proposal should be evaluated carefully against existing systems research. Distributed systems have long used namespaces, security domains, capabilities, containers, process isolation, access-control policies, provenance graphs, and execution contexts. SCW claims novelty for none of these individually. The proposed contribution is the composition of these ideas around autonomous-agent execution into a first-class addressable semantic

execution-space abstraction. It is worth being precise about what is and is not being claimed. The question is not “has anyone ever isolated a process?” — clearly they have. Nor “has anyone ever tracked provenance?” — they have. Nor “have multi-agent systems ever used separate contexts?” — they have. The question is whether an explicit abstraction that binds context, identity, capability, environment, provenance, and agent transitions into one addressable coordinate provides a useful systems primitive for controlling autonomous multi-agent execution — and that proposition is empirically testable rather than assumed. Understood this way, SCWs are complementary to the rest of the stack, naming two responsibilities — the SCW boundary and the SCW⁰ control layer — that the stack otherwise leaves implicit. A sandbox is not a context router; a context window is not a capability policy; an agent identity is not an execution boundary; an MCP connection is not authorization; a tool declaration is not necessarily provenance; and an API session is not necessarily a complete execution-space abstraction. SCW exists in the relationships among these components.

SCW and SCW⁰ name two responsibilities the stack otherwise leaves implicit

Human	oversight & authorization where required
Monitoring	operational observation
Operating system	low-level resource enforcement
Sandbox	physical execution restrictions
SCW⁰	cross-SCW control, routing, policy, provenance
SCW	context / execution-space binding & boundary semantics
MCP	standardized external-system connectivity
Agents API / harness	lifecycle, orchestration, context, tools, delegation
Agent	task-oriented autonomous behavior
Language model	reasoning and generation

Figure 12. SCW and SCW⁰ placed among the layers of a contemporary agent stack. A sandbox is not a context router; a context window is not a capability policy; an MCP connection is not authorization.

13.1 Limitations

SCWs do not solve AI alignment. They do not guarantee that an agent understands a task, that an authorized action is desirable, that an underlying sandbox is secure, or that outcomes are correctly verified; they do not eliminate prompt injection, malicious tools, or compromised infrastructure; and they may introduce significant implementation complexity. They also create a new critical component in SCW⁰ itself: a failure there could affect many execution spaces, so SCW⁰ must be treated as security-sensitive infrastructure. For that reason the architecture must specify its failure behavior explicitly rather than leave it to defaults.

Fail-state	Intended behavior
fail-open	transitions proceed without full evaluation — generally unacceptable for privileged operations
fail-closed	transitions are denied when the control layer cannot evaluate them
fail-safe	the system falls back to a known-safe bounded configuration
degraded	reduced capability with explicit loss of some control or observability
unavailable	execution is suspended pending control-layer recovery
human-review	transitions are escalated to a person before proceeding

13.2 Future research and falsifiability

Several areas require investigation: a formal SCW semantics for creation, inheritance, nesting, admission, capability transfer, delegation, termination, and provenance; declarative policy languages (“SCW-A may send summaries to SCW-B but may not transfer raw customer records”); automatic context classification by sensitivity, authority, origin, destination, trust, and task relevance; execution graphs in which SCWs are nodes, capabilities and resources are objects, and delegations and policy decisions are edges; large-scale evaluation across growing agent populations; harness integration across multiple runtimes; physical-enforcement mappings to containers, virtual machines, microVMs, OS security domains, and cloud workloads; and human-oversight policy for when transitions execute automatically, require approval, trigger verification, or terminate a workflow. The hypothesis is explicitly falsifiable: it would be weakened if existing harnesses already provide equivalent explicit execution-space semantics, if SCW metadata yields no measurable improvement in provenance or policy attribution, if context partitioning does not reduce leakage, if policy overhead degrades productivity without safety benefit, if existing sandbox and capability systems express the same abstraction without an SCW-level construct, or if large-scale systems cannot practically maintain SCW state.

13.3 Discussion

The central architectural claim is deliberately narrower than a claim that SCWs constitute a complete solution to agent safety. The claim is that autonomous-agent execution may benefit from an explicit address-space abstraction that connects context to execution, and this becomes more important as systems move from single-agent interactions toward persistent, multi-agent execution. A language model operates on context; an agent acts through capabilities; a harness coordinates execution; a sandbox constrains physical resources; a protocol connects external systems — but autonomous behavior emerges from their combination, so the system’s safety boundary cannot necessarily be described by any single component. The SCW abstraction attempts to provide a common coordinate system across them.

In that coordinate system, several familiar objects are re-read as facets of an execution space. Context is not merely text; it is state associated with an execution space. A capability is not merely a tool; it is an authorization relationship associated with an execution space. A sandbox is not merely a container; it is a physical enforcement substrate associated with an execution space. Delegation is not merely spawning another model; it is a transition between execution spaces. Observability is not merely logging model outputs; it is reconstructing the evolution of execution spaces and their relationships. Taken together, these lead to a broader systems

principle that the architecture is built to express.

The broader principle

Autonomy should occur inside explicit execution spaces whose boundaries are addressable, observable, and enforceable. SCW is one proposed implementation of that principle — not the only possible one, and valuable only insofar as it can be built, measured, and, if it fails to help, revised or rejected on the evidence.

13.4 Conclusion

AI agents are becoming systems rather than isolated model calls: they maintain context, invoke tools, access external systems, execute code, persist state, operate in sandboxes, delegate work, and communicate with other agents. As these capabilities expand, safety increasingly becomes a question of execution-space control. Modern infrastructure already supplies major pieces — orchestration, context management, subagents, tools, MCP, and environments from the harness side, and physical boundaries from the sandbox side. Structured Context Worlds propose a complementary abstraction that makes the agent’s execution space explicit by binding context, identity, capabilities, resources, environment, provenance, and permitted transitions into a single addressable unit, with SCW⁰ as the supervisory layer that creates, routes, observes, and governs those spaces. The architecture replaces neither the harness nor the sandbox nor the model; it provides the structural layer connecting them.

The research hypothesis, in one line

If autonomous agents are treated as actors operating inside explicit, addressable execution spaces rather than merely as model instances with tools, then context routing, capability control, provenance, delegation, observability, and system-level safety can become properties of the execution architecture itself. Whether that produces measurable safety and productivity benefits remains an empirical question — and that question is the purpose of the SCW research program.

Author statement

Maxey0 is the AI systems architecture and research framework through which the SCW concept is developed, and is credited as lead author of this work. Mike Cohan is the human partner and co-author responsible for the research partnership, engineering direction, system development, interpretation, and publication. Claude (Opus 4.8) served as Co-Editor for this edition: it expanded the paper’s listed and condensed sections into full prose, designed the twelve explanatory figures and the summary tables, and restructured the manuscript into this format, without introducing new architectural claims or external research beyond the source material. Contact: human@maxey0.com.

Note on sources and claims

This paper describes the Maxey0 research architecture and proposes SCWs and SCW⁰ as an execution-space abstraction for autonomous agents. The authors do not represent the proposal as an established industry standard or a demonstrated production safety system. Where the paper discusses external agent infrastructure, the statements are based on publicly available documentation and publications; the source manuscript cites those references as of September 2026. Where the paper describes Maxey0 concepts — SCWs, SCW⁰, context admission, delegation transitions, and related architecture — those descriptions represent the research program and proposed architecture unless an external implementation or result is explicitly identified. No experimental result is claimed unless supported by a documented experiment. Figures are architectural; the one quantitative-looking figure (the controllable frontier) is explicitly labeled conceptual and carries no measured data.

References

- [1] OpenAI. Introducing the Agents API. September 10, 2026.
- [2] OpenAI. Agents API — Overview. OpenAI API documentation. Accessed September 2026.
- [3] OpenAI. Agents API — Architecture. OpenAI API documentation. Accessed September 2026.
- [4] OpenAI. Agents API — Multi-agent. OpenAI API documentation. Accessed September 2026.
- [5] OpenAI. Agents API — Environments and Sandboxes. OpenAI API documentation. Accessed September 2026.
- [6] OpenAI. Sandbox Security. OpenAI API documentation. Accessed September 2026.
- [7] OpenAI. Running Codex Safely at OpenAI. May 8, 2026.
- [8] OpenAI. Building a Safe, Effective Sandbox to Enable Codex on Windows. May 13, 2026.
- [9] Model Context Protocol. What is the Model Context Protocol? 2026 specification / documentation.
- [10] Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.
- [11] Prior work in operating-system isolation, capability security, distributed-system namespaces, workflow provenance, multi-agent coordination, and information-flow control provides the foundational mechanisms underlying the SCW proposal. The contribution of this paper is their proposed composition as an explicit semantic execution-space abstraction for autonomous agents, not the invention of those mechanisms individually.
- [12] Maxey0. Maxey0 System One: A Provider-Neutral Decision Layer for Structured Context Worlds. Companion Maxey0 research paper (semantic decision layer operating over SCWs).